DAT602 Database application development

Assignment 1.

By Zandi Galpin.

Table of Contents

Task 1, Description of game, storyboarding and user stories: 2

Description: 2

Storyboards: 4

Task 2, logical entity relationship diagram depicting the planned database: 26

Task 3, CRUD table as part of analysis for the game system: 27

Task 4, DDL SQL script to create tables and relations between tables to support the game: 29

5. Create a prototype C# .net application using Visual Studio: 29

Task 1, Description of game, storyboarding and user stories:

Description:

This game will be titled Ultimate Goose Fighter. It is a semi-top-down (characters are shown from the side, but the environment is top-down) multiplayer combat game in which players play as geese. Each goose has a health bar above it visible to all players. Players will move either with the arrow keys or WASD. The player can attack with the space bar. Attacking causes the goose to extend its neck and peck forwards one tile in the direction they are facing. A goose's beak must be in contact with another goose to do damage. After being damaged, a goose is invincible for 0.5 seconds. When a goose's health is depleted, they respawn in a different location on the map, and they are invincible for 5 seconds. While a goose is invincible their sprite flashes. The player who killed them has their score increased by 1.

All geese have 4,000 max health and deal 40 base damage (*these numbers and others are subject to change*).

Each goose has 1 inventory slot (holding item in mouth). Holdable items can be picked up by coming into contact with the goose's beak or body (either by using the attack button or walking onto the tile the item is on). Items spawn randomly. While holding an item, use the space bar (attack button) to use the item or attack with it. Press L or G to drop the item.

Holdable items can include the following:

- Baseball bat (+50% attack damage).
- Pellet Gun (-50% attack damage, ranged attack, disappears after ammo is depleted, random amount of ammo from 3 to 10, bullets travel in the direction the goose is facing when shot until they hit something, if a goose picks up another pellet gun while they still have ammo then more ammo is added).
- Shield (-50% incoming damage, -25% outgoing damage).
- Grenade (when activated, is thrown straight forward 3 tiles in the direction the player is facing, explodes after 1 second, doing 500% damage in a square shape of 9 tiles).

Specific spawn rates (percentages) are to be determined.

When a goose drops an item, that item disappears and cannot be picked back up again. Consumables will also randomly spawn on tiles. A goose cannot eat consumables while it has an item in its mouth (this is why items disappear when dropped and can't be picked up again. If a player has a powerful item, they should think twice before dropping it to eat consumables). Consumables are picked in the same way as items are picked up (coming into contact with beak or body) and you can either press the space bar to consume them or L or G to drop them.

Consumable items can include the following:

- Peas (when eaten, regain 200 health).
- Banana (spawns less, when eaten, regain 400 health).
- Grapes (spawns rarely, when eaten, the player gains invincibility for the next 5 seconds).
- Watermelon (spawns rarely, when eaten, regain 1,000 health).

Specific spawn rates (percentages) are to be determined.

Each player will have a home nest that spawns under them when they first spawn (or when they respawn). While a player is standing on their nest, they cannot be attacked (flashing invulnerability) but can still attack others. If a player drops an item or consumable on their nest, it goes into the nests inventory, which has 9 slots. If the nest's inventory is full, clicking the drop buttons (L or G) will not do anything. While standing on the nest, the nest's inventory will be open. Items in the nest will be listed with images. Clicking on an item will remove it from the inventory and place it in the goose's inventory (if there is not already an item in it). If a player's health is depleted, their nest despawns and all items in the nest are lost. The player respawns and a new nest is spawned under them.

If a player leaves the game, their nest's position is saved, any items in the nest are saved, their goose's position on the map is saved, their health is saved, their score is saved, and any item they may be holding is saved.

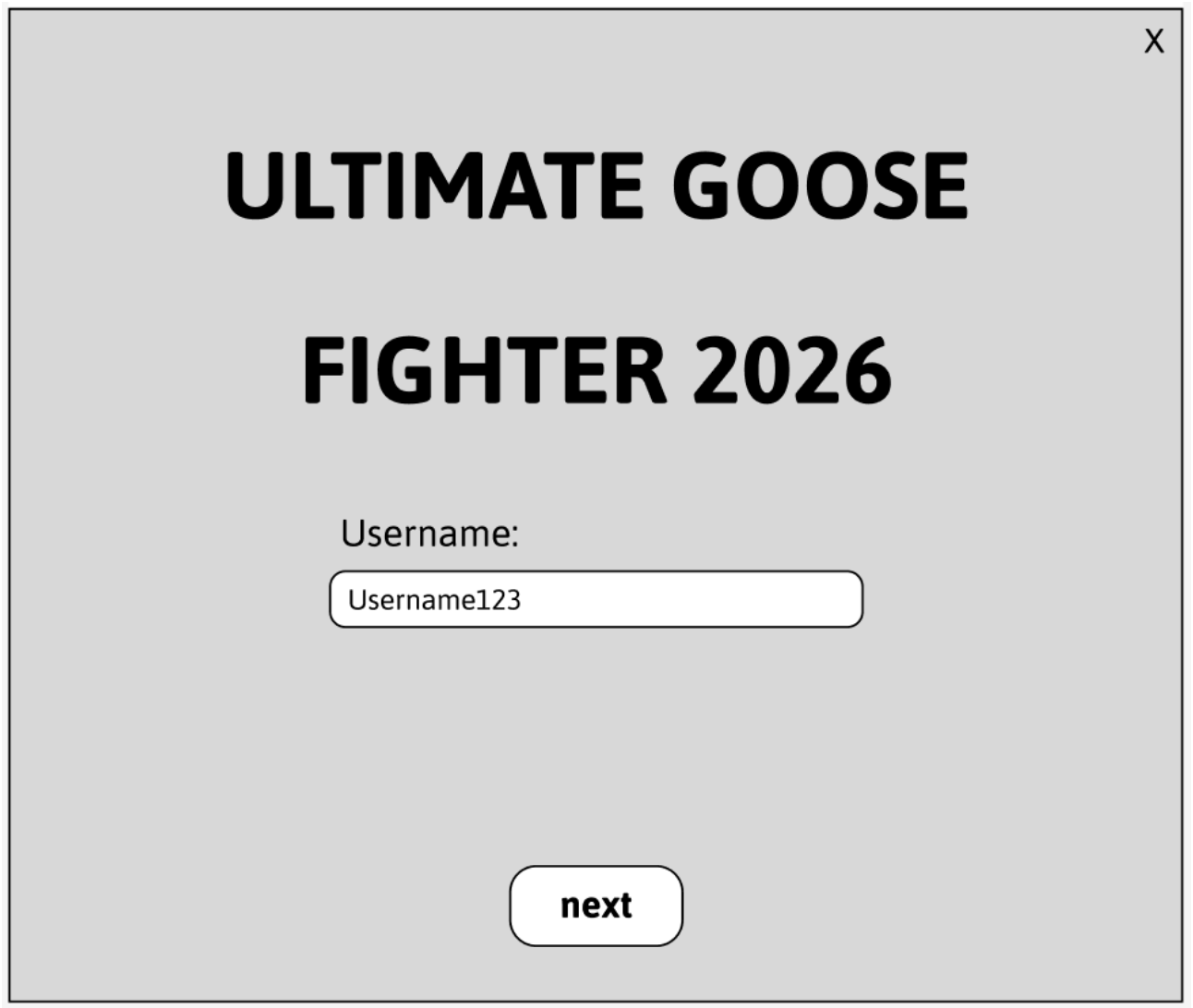
Items and consumables will not spawn unless there are at least 2 players online.

When a new account is created the default score is 0.

There will be no more than 1 NPC on the map at all times. The NPC will be a grey goose which wanders randomly. It has 1,000 health and a health bar indicating its health. When defeated, it drops an item or consumable completely randomly (ignores spawn rates and instead treats the probability of all items/consumables equally, therefore rare items are more likely to drop). When the NPC dies, a timer of 1 minute begins. After 1 minute another NPC spawns in a random location.

Storyboards:

Login screen, storyboard 1 – requirement 1/2:



A user first opening the application is brought to this screen with a username field, a "Next" button, an "X" in the top right to close the application, and the title "ULTIMATE GOOSE FIGHTER 2026". They type in a username and click the "next" button. If the username exists in the database, the screen changes to storyboard 2. Clicking "Next" runs a SELECT against PLAYER on Username to check whether the row exists, which determines whether the user goes to storyboard 2 or 3.

If the username doesn't exist in the database, the screen changes to storyboard 3.

Login screen (existing user), storyboard 2 – requirement 1:



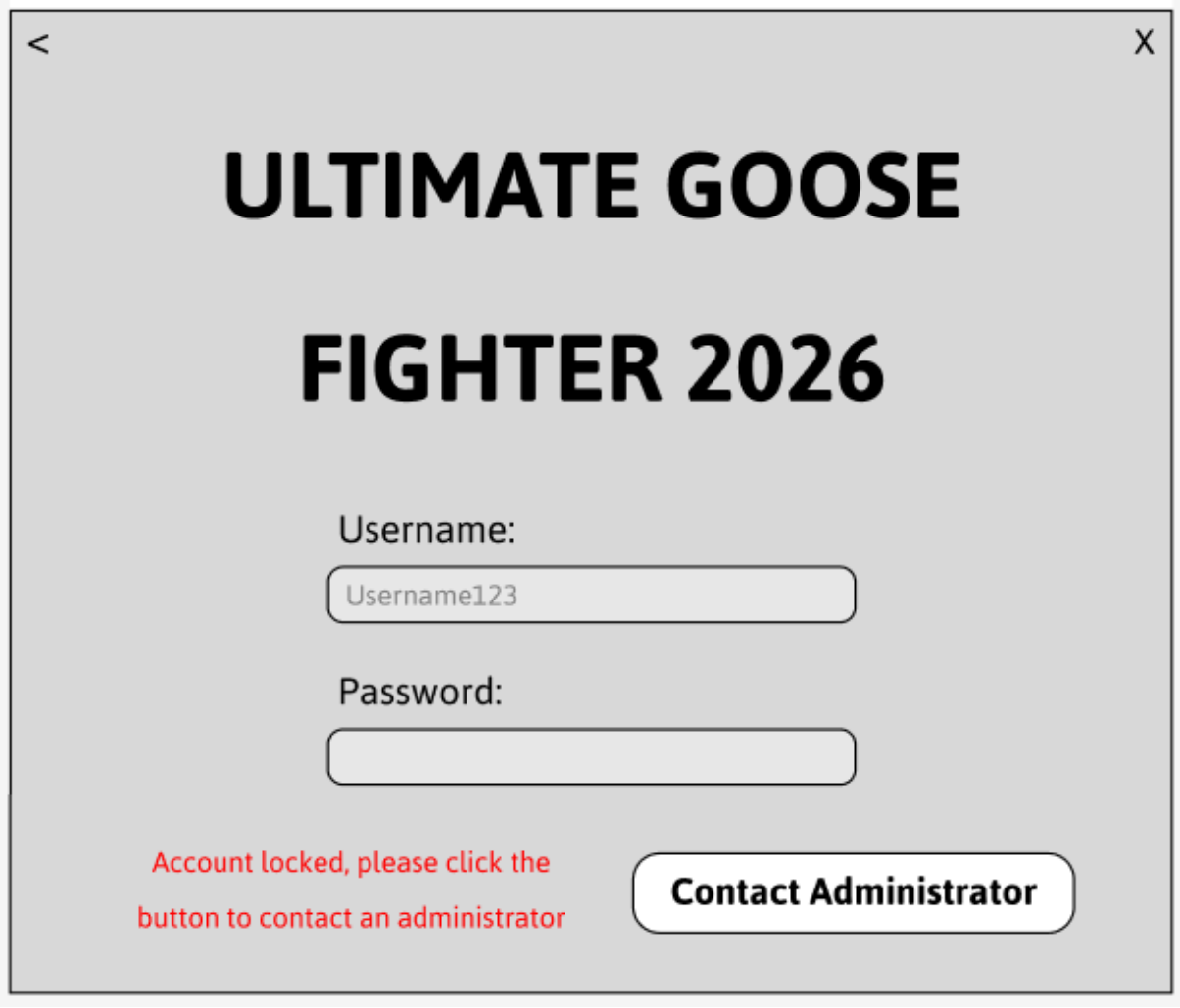
When a user has entered an existing username in storyboard 1, they are brought to here. The username is greyed out and can no longer be changed. A “Login” button appears in place of the “Next” button and a “Password” field appears under the Username field (starting empty until typed in, appears as asterisks), along with a back arrow in the upper left corner. If the back arrow is clicked, this directs the user back to the screen in storyboard 1. The user types a password in and clicks the “Login” button. If the password is correct and the user is a regular user, the screen changes to Storyboard 12. If the password is correct and the user is an admin, the screen changes to Storyboard 17. If the user is a new regular user (created by an admin but hasn’t logged in before), they are taken to storyboard 5. If the password is incorrect, the password field is cleared and the text “Incorrect password” appears under the password field, and they are given a chance to try again. After 5 incorrect attempts, the screen changes to storyboard 4. Password is checked against the stored PLAYER’s username. A failed attempt increases FailedLoginAttempts by 1. A success resets it to 0. If it gets to 5 then IsLockedOut is set to 1.

Registration screen (new user), storyboard 3 – requirement 2:



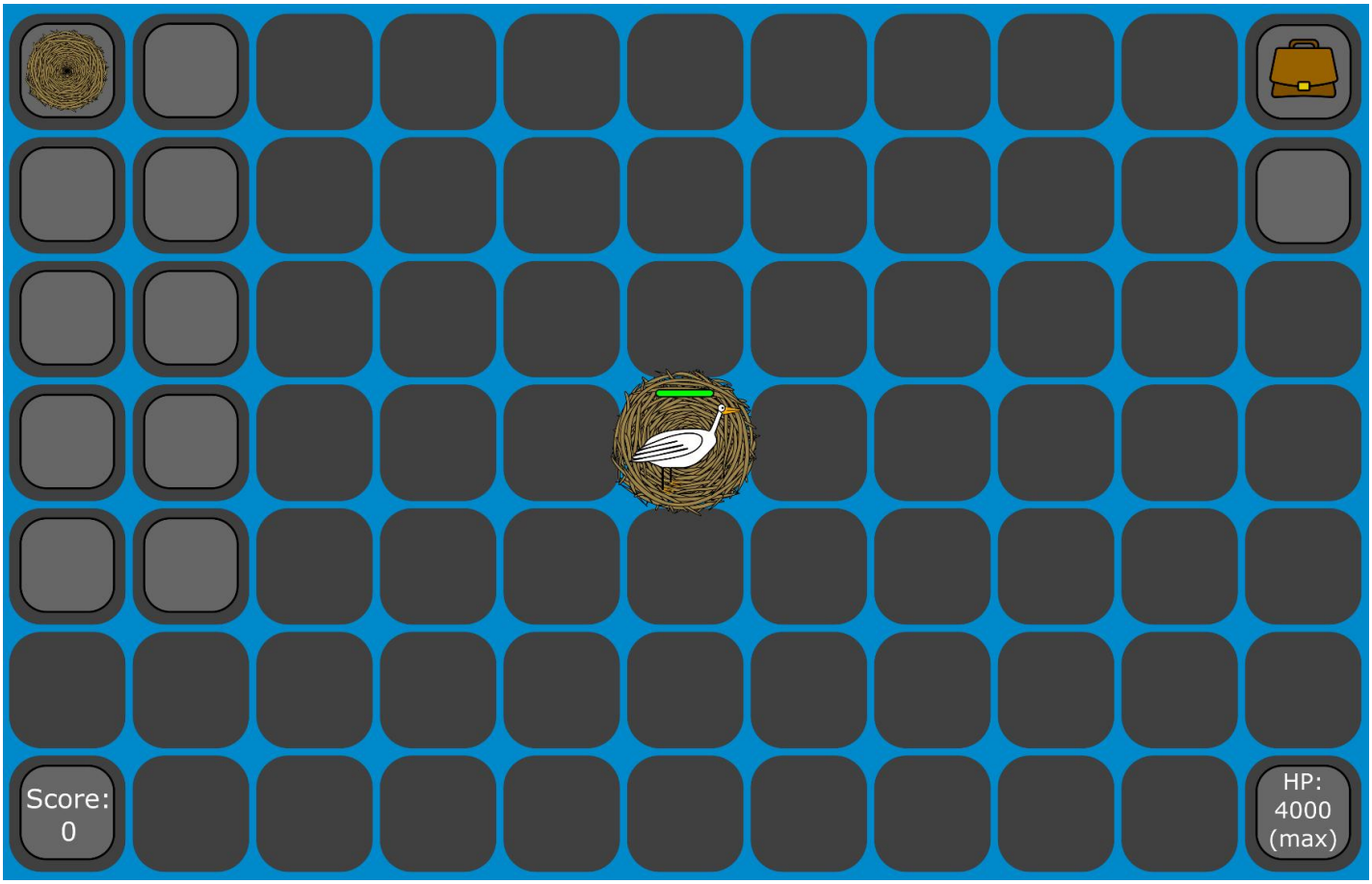
When a user has entered a name that doesn't currently exist in the database in storyboard 1, they are brought to here. The username is greyed out and can no longer be changed. A "Create account" button appears in place of the "Next" button. An "Email" field appears above the Username field, and a "Password" field appears underneath the Username field (both starting empty until typed in, password text appears as asterisks), along with a back arrow in the upper left corner. If the back arrow is clicked, this directs the user back to the screen in storyboard 1. If the user leaves either of the entries empty, clicking the "Create account" button won't work. If the user has typed in them, clicking the button will change the screen to storyboard 5. "Create account" inserts a new PLAYER row with the entered email, username and password, a score of 0, IsAdmin set to 0, IsLockedOut set to 0.

Lockout screen (5 incorrect attempts), storyboard 4 – requirement 1:

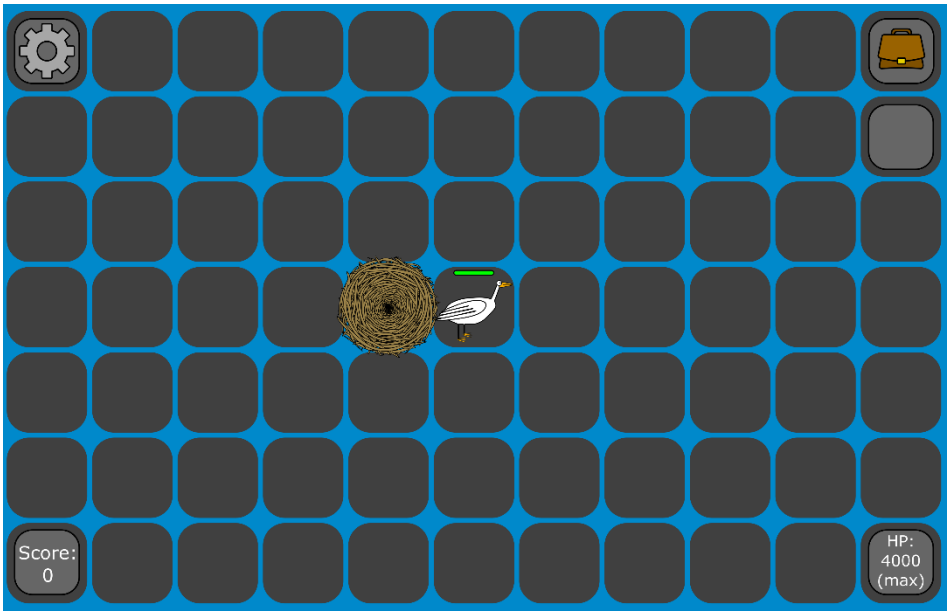


When a user enters an incorrect password 5 times on storyboard 2, the screen changes to this. The password is cleared and the field is greyed out and can no longer be typed in. The text “Account locked, please click the button to contact an administrator” appears underneath in red, and the “Login” button disappears and is replaced with a “Contact Administrator” button. If this button is clicked it displays an admin’s contact info. This screen reflects PLAYER.IsLockedOut = 1. Only an admin unticking “Account locked?” on storyboard 11 can set it back to 0.

Game start screen for new user, storyboard 5 – requirement 6:



When a user creates a new account (from storyboard 3) they are brought here. They spawn in on a random tile on the map with a nest under them. In this case all tiles on the screen are filled in. This doesn't have to be the case. A player can spawn with their nest on a tile on the edge of the map. The score, HP and inventory overlay will always be on screen. The nest overlay will only be on screen while standing on the nest. The user then presses the "D" key on their keyboard, moving them to the next tile on the right, as seen here. Also, when the player is not on their nest, the settings icon will appear as an overlay. This stays on screen at all times that the user is not standing on their home nest. The user clicks this icon and is brought to storyboard 7. A NEST row is inserted (linked to the new PlayerID and a randomly chosen TileID). PLAYER.CurrentTileID is updated to that tile.



Settings screen (regular user), storyboard 7 – requirement 3:



Anytime a regular user (non admin) clicks the settings icon while they are in the game (in this case from storyboard 5 or 12), they will be brought to this screen. While on this screen their goose will be invincible. There will be a “Return to title screen” button which when clicked will send the user back to storyboard 1 (logging them out). There is also a “DELETE account” button. When clicked, it will prompt the user asking them if they are sure they want to delete their account, with “yes” and “no” options. If the user clicks the “yes” button, their account will be deleted and removed from the database, and they will be sent to storyboard 1. Clicking the back arrow on the top left will return them to the game (storyboard 5 or 12) and they will no longer be invincible. “DELETE account” removes the PLAYER row, and deletes their NEST row and any ITEMs they were holding or storing. “Return to title screen” sets IsOnline to 0 for the user’s PLAYER row.

Settings screen (admin), storyboard 8 – requirement 11, segway to requirements 12, 13 and 14:



When a user who is an administrator clicks the settings button (in this case from storyboard 17), they will be brought to here. This screen is the same as in storyboard 7 except the “DELETE account” button is replaced with an “Edit players” button, plus there is a new “KILL current game” button. The “Return to title screen” button, when clicked by the user, sends them back to storyboard 1 (logging them out). The user (admin) clicks the “Edit players” button and they are brought to storyboard 9. Clicking the back arrow on the top left will return them to the game (in this case storyboard 17). If the user had instead clicked the “KILL current game” button, they would be prompted with text saying “Are you sure” and given the options “Yes” and No”. If they click no, the text goes away. If they click yes, the current game is killed and all players are kicked offline. Clicking the “Kill current game” button and confirming runs a bulk UPDATE against the PLAYER table. IsOnline is set to 0 for every row that is set to 1. Does not change anything else. “Return to title screen” sets IsOnline to 0 for the user’s PLAYER row.

Edit players screen (admin), storyboard 9 – requirement 14, segway to 12 and 13:



When a user who is an administrator on storyboard 8 clicks the “Edit players” button they are brought to here. There is a back arrow in the same place as before which when the user clicks it, returns to the settings screen (storyboard 8). There is an “Add new player” button which when clicked takes the user to storyboard 10. Under this button is a list of existing players. In this screenshot, the user has selected (clicked) the player “McPlayer Twoington”. Under the list of existing players there are two buttons. A player must be selected for either of them to be clicked. The first button is the “Edit selected player" button, which when clicked brings the user to storyboard 11. The second button is the “Delete selected player” button, which when clicked, removes the selected player and any related player information from the database, along with removing it from the list on screen. The list of players is populated by selecting all usernames from the PLAYER table. “Delete selected player” does the same delete as in storyboard 7.

Add new player screen (admin), storyboard 10 – requirement 12:



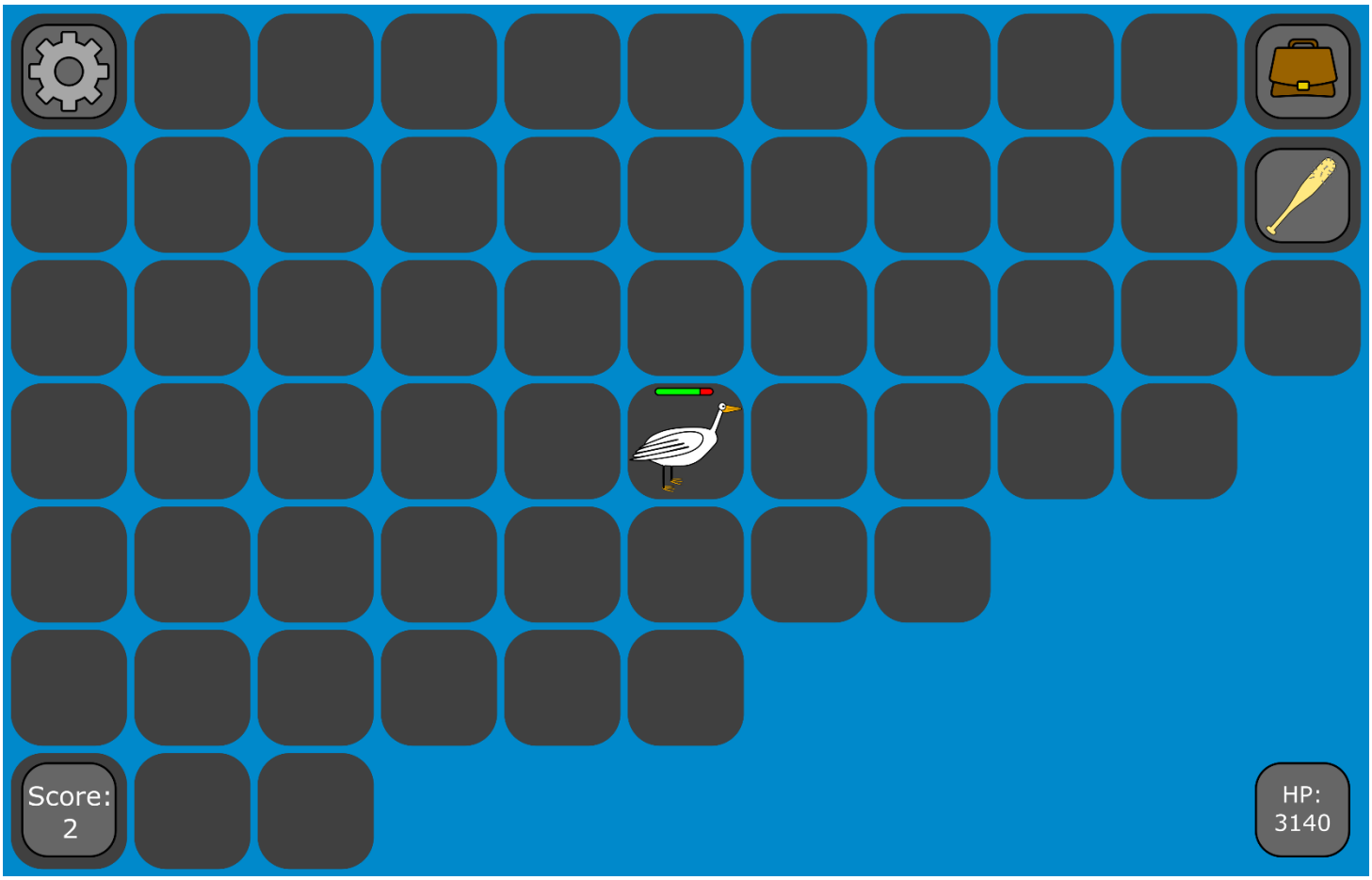
While on storyboard 9, if the user clicks the “Add player” button, they are brought to here. The back arrow, when clicked by the user, returns them to storyboard 9 without saving any details entered. There are “Email”, “Username” and “Password” fields like in storyboard 3, except here the username field is not greyed out as it needs to be entered along with the other info. These fields cannot be left blank before submitting. There is also an “Admin?” tick box which is by default unticked. If the user ticks this and submits, the player will be given administrator privileges. The user submits by clicking the “Add player to database” button at the bottom. This adds the player information to the database, then sends the user back to storyboard 9, where the new player is listed along with the old ones. Same INSERT as storyboard 3, but IsAdmin is set based on the tick box instead of automatically being set to 0.

Edit selected player screen (admin), storyboard 11 – requirement 13:



While on storyboard 9, if the user (who had selected McPlayer Twoington) clicks the "Edit selected player" button, they are brought to this screen. This has "Email", "Username", "Password" and "Score" fields which are all stored in plaintext, contain the players current information from the database, and are able to be edited. There is also an "Admin?" tick box and "Account locked?" tick box which are either already ticked or unticked depending on the data in the database reflecting each of them (Booleans). The back arrow, when clicked by the user, returns them to storyboard 9 without saving any details which have been altered. The user edits some details about McPlayer Twoington, including changing his username to "Mcplayerington Two". There is an "Update player" button at the bottom which, when clicked by the user, updates the players information in the database to what has been edited on this screen by the user, then sends the user back to storyboard 9, which appears the same, except instead of "McPlayer Twoington", "Mcplayerington Two" is now listed (not to be implemented in this project, but a good idea would be to send the player an email about their username and/or password getting changed). "Update player" runs an UPDATE PLAYER SET … WHERE PlayerID = … for all the edited fields, including IsAdmin and IsLockedOut.

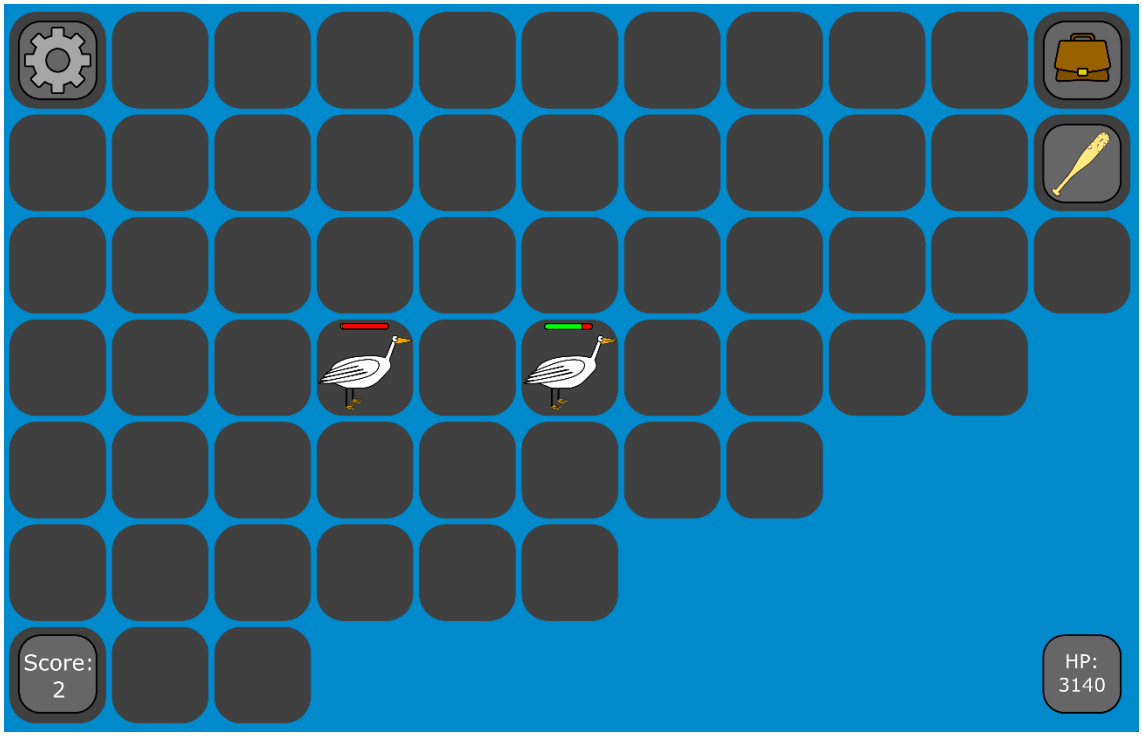
Game start screen for returning user, storyboard 12, segway to requirement 7:



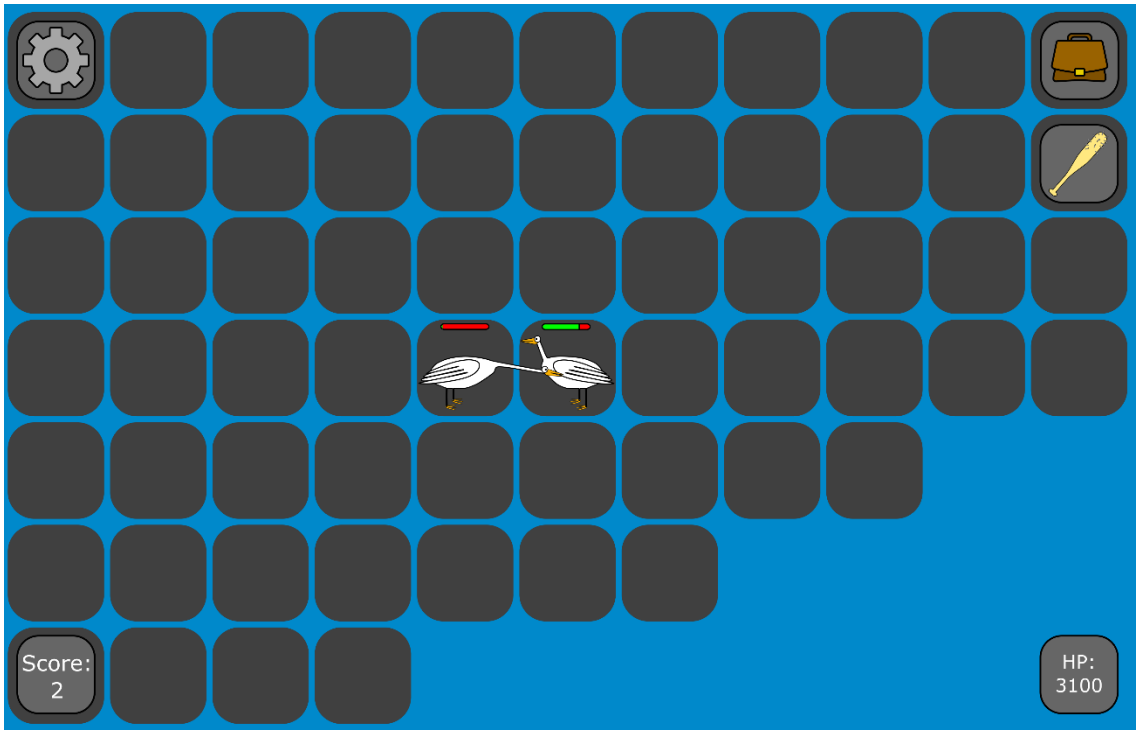
On storyboard 2, if the user clicks the “Login” button with correct credentials, they will be brought to wherever they were when they last left the game, in whatever state they were in with whatever item they had. In this case we can see that the user had 3140 HP, a score of 2 and a baseball bat when they last logged out. If the user clicks the settings icon in the top left, they will be brought to storyboard 7. In this case the user doesn’t click this and continues to storyboard 13. On login, PLAYER’s saved CurrentTileID, Health, Score and HeldItemID are read to restore the state they had when they left, and IsOnline gets set to 1.

User defeating another player and gaining score, storyboard 13, requirement 7:

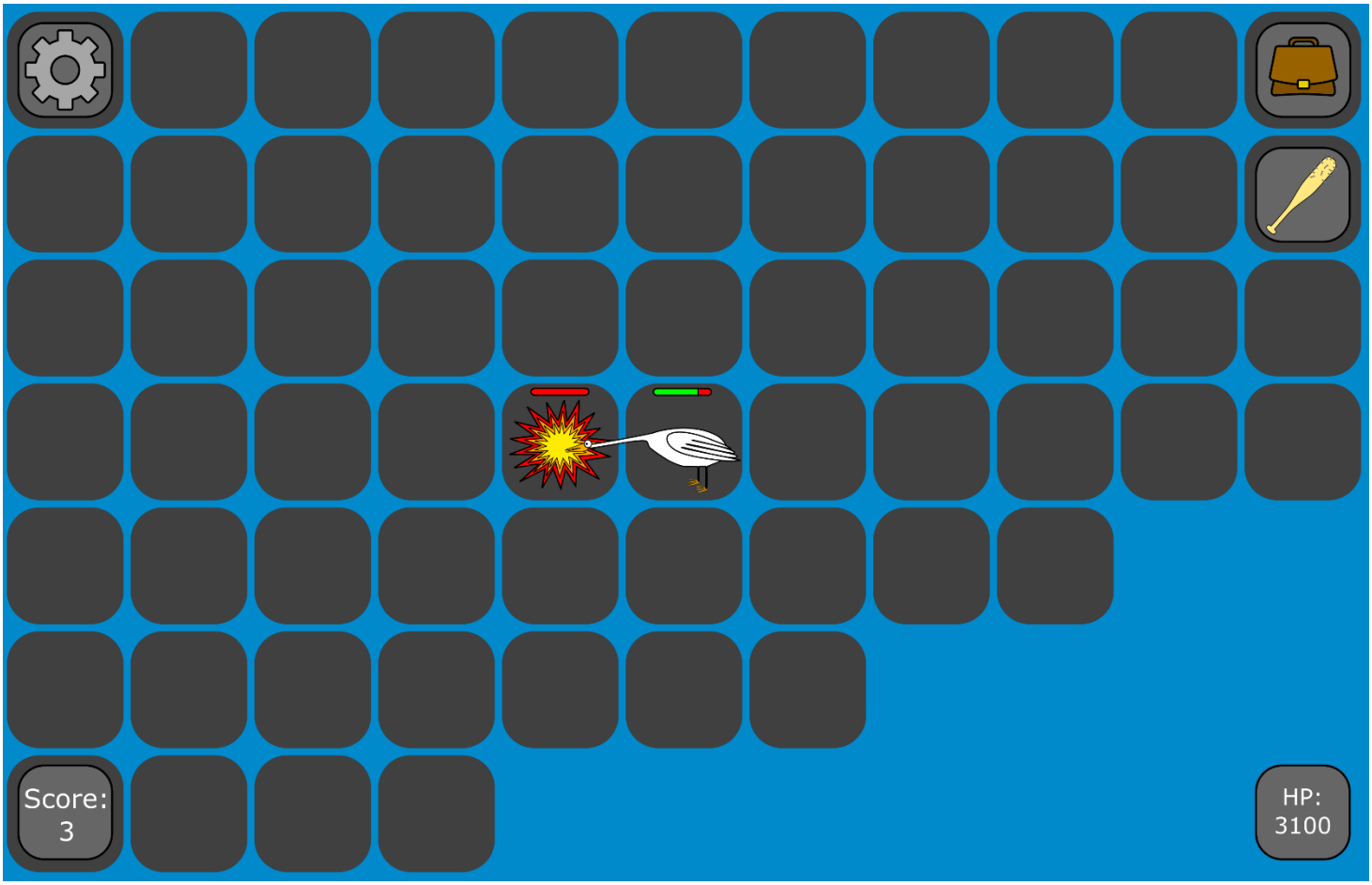
In this case the user stays still (no inputs) and another player comes on screen and moves across to them until they are 2 tiles away, as seen below.



The user then clicks the left arrow key, moving one tile to the left. And the other player clicks the space bar, attacking forward onto the tile the user is on. This decreases the user's health by 40, as shown below.



The user then clicks the space button, attacking to the left where the other player is standing. With the baseball bat this deals 60 damage to the player. The player had 50 health before being attacked so upon getting hit they are defeated. The remaining green on their health bar disappears and it turns entirely red, and their goose sprite is replaced by an explosion sprite. The user's score then increases by 1 as can be seen below. The other player's home nest is deleted.



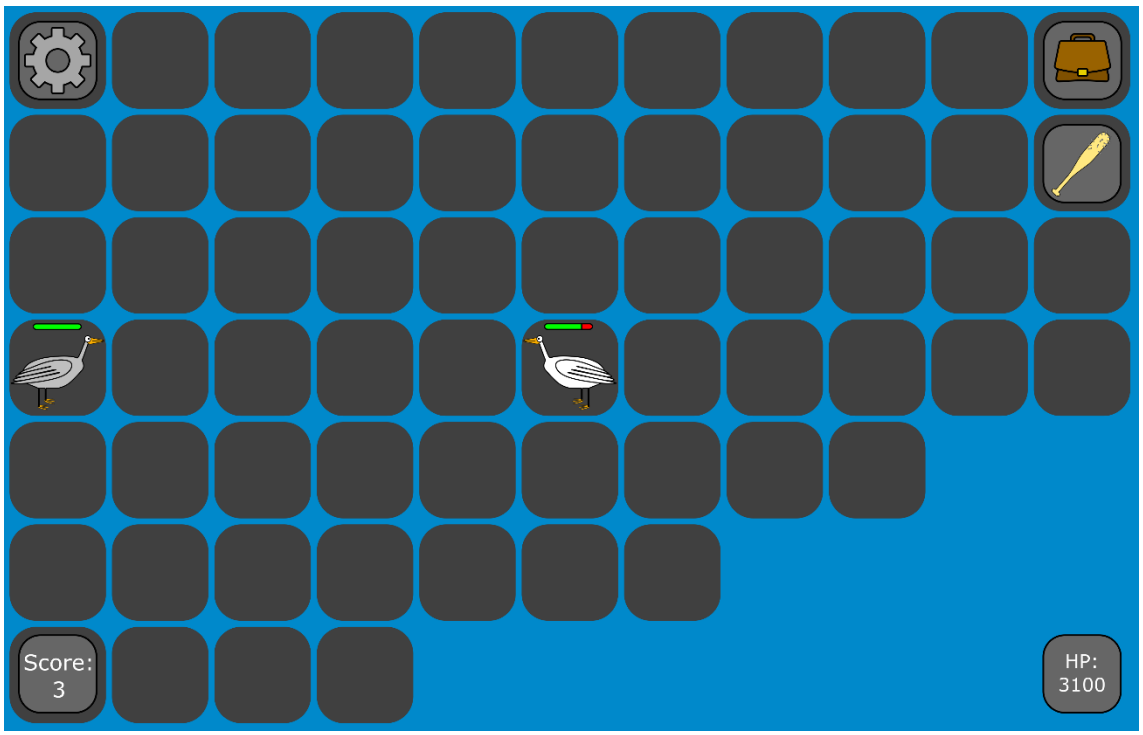
The explosion and empty health bar then would disappear, the user's sprite would change from the attack sprite to the standing left sprite, and the other player would respawn somewhere else on the map with a new nest.

Damage taken updates PLAYER.Health. When defeated, the attackers SCORE is increased by 1. The defeated player's NEST row is deleted, and their PLAYER row gets a new random CurrentTileID, a new NestID and a new NEST row is added with a random TileID. The player's HEALTH is reset to 4000 and IsInvincible is set to 1 and InvincibleUntil is set to 5. 5 seconds later IsInvincible is set to 0.

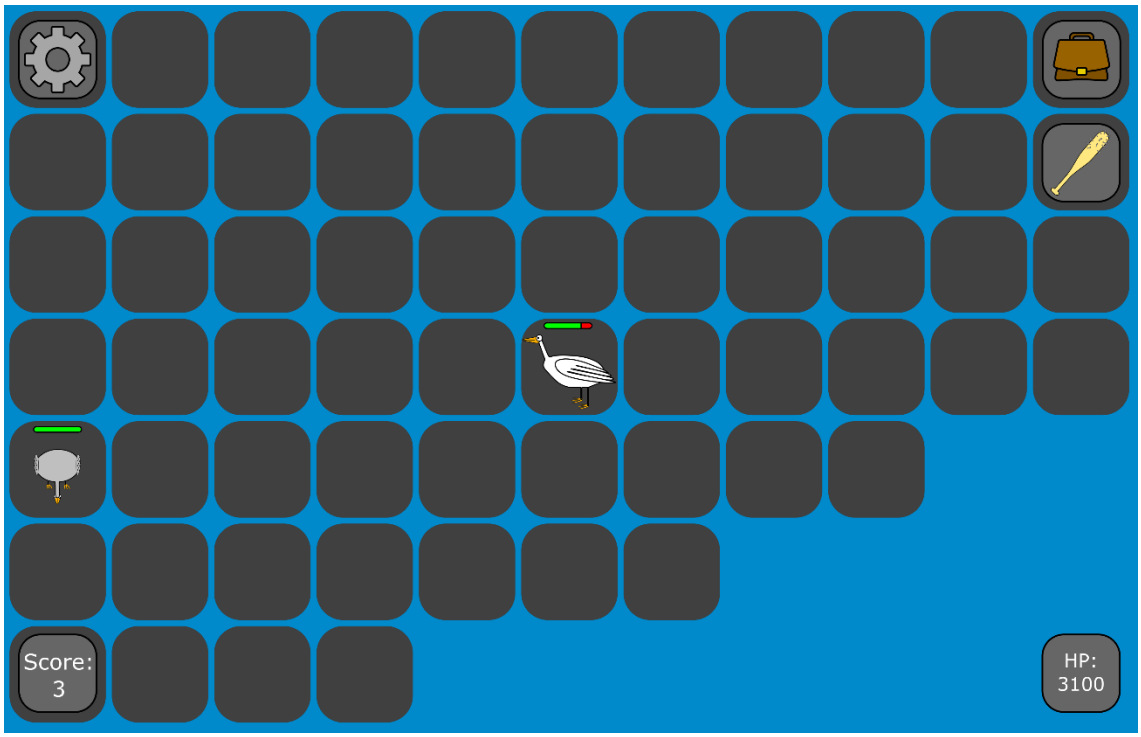
Continue to storyboard 14.

NPC walks on screen and wanders around, storyboard 14, requirement 9:

The user stands still and doesn't do anything (no input). The grey goose (an NPC with 1000 health, doesn't attack the player, just wanders randomly) moves on screen (from the left). If defeated he will drop a random item or consumable on the tile he is defeated on (explosion sprite will play like the other storyboard, but the user will not get score).



After a random amount of time (between 1 and 5 seconds) the grey goose then moves again one tile in a random direction. In this case it moves down (shown below).

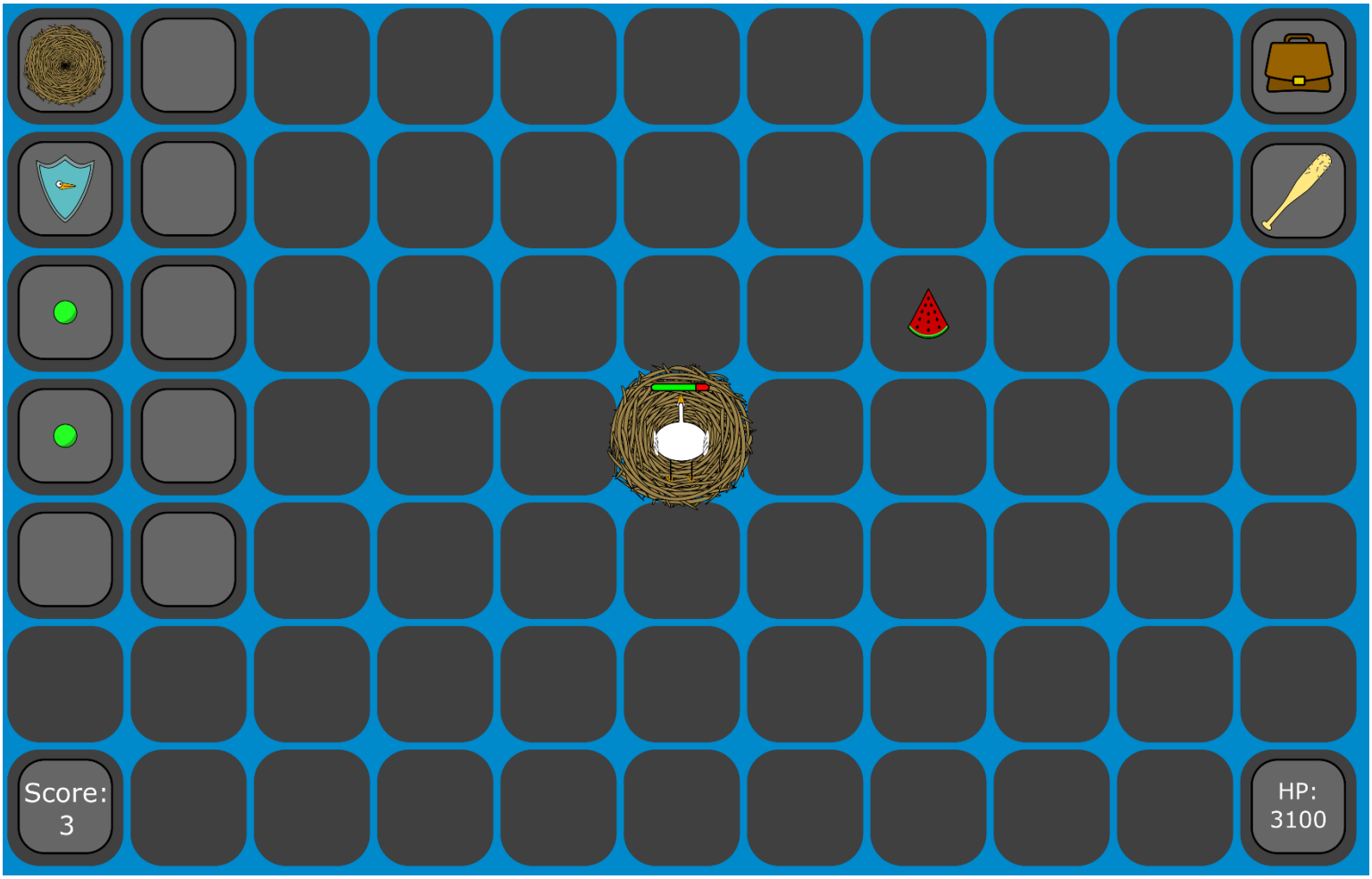


Each random move updates NPC.TileID. When defeated, IsAlive is set to 0 and a new ITEM row is inserted at the last tile the NPC was on with a randomly chosen ItemTypeID. After the 1 minute timer, a new NPC row is placed at a random tile.

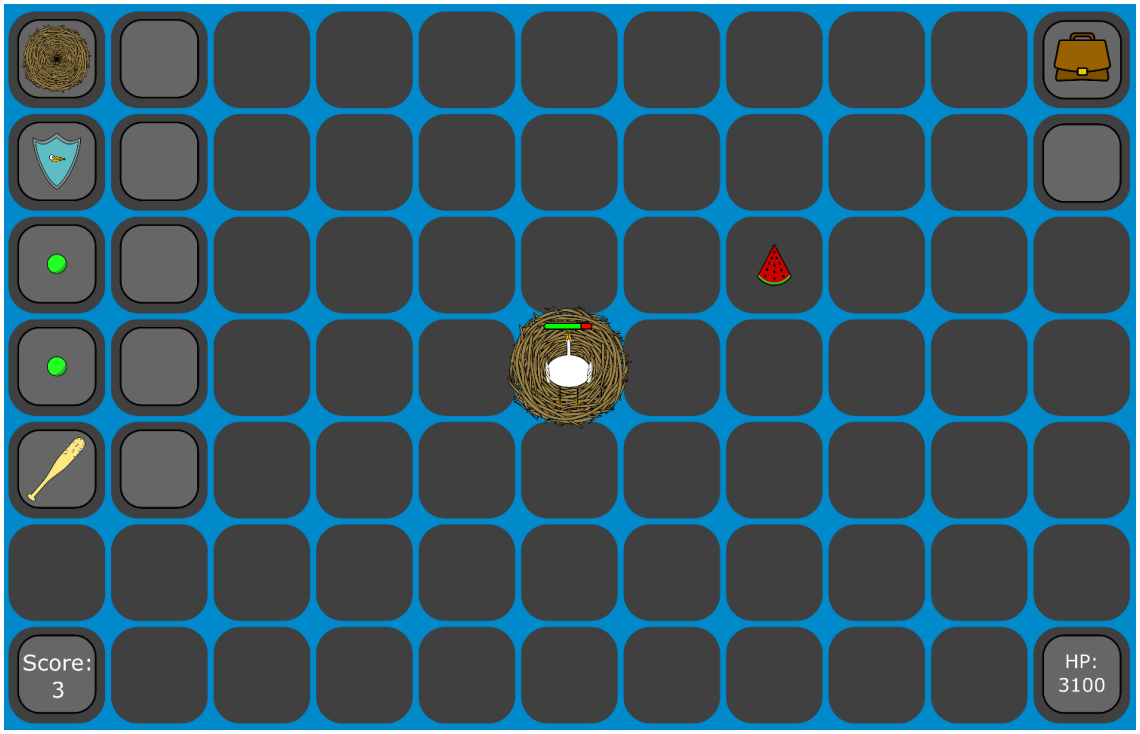
Continue to storyboard 15.

User stores current item in nest, collects new item, storyboard 15, requirement 8:

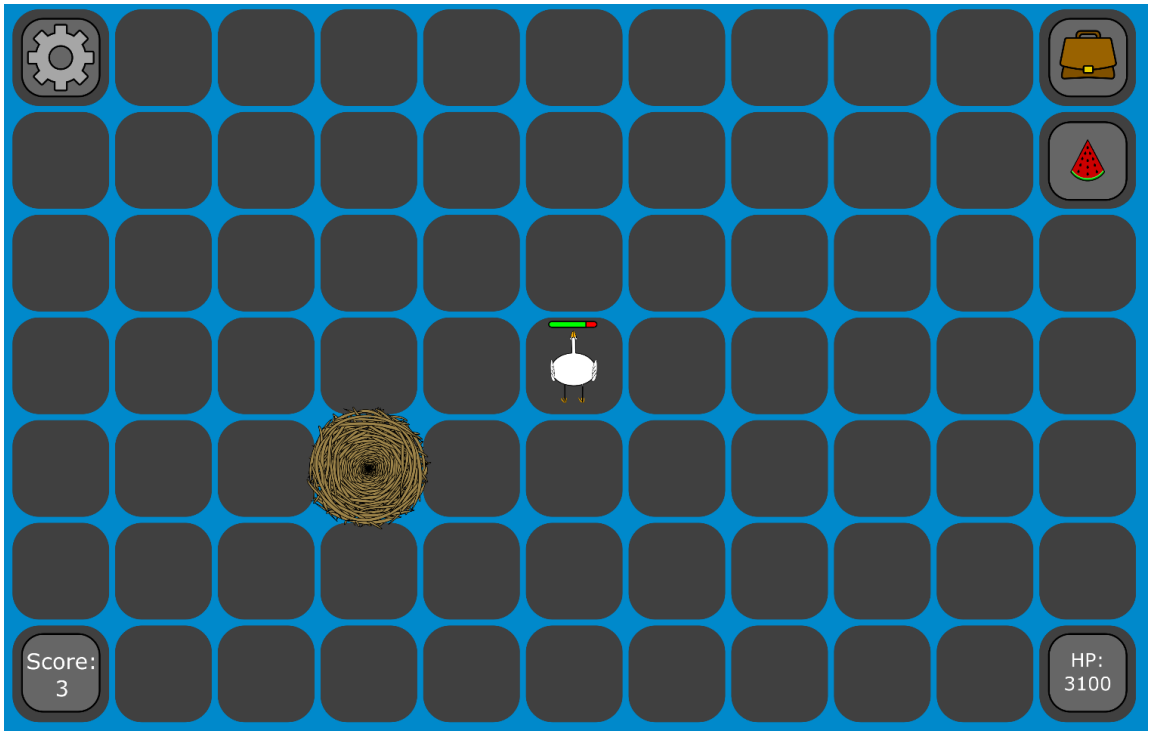
The user walks up towards their nest and notices a watermelon which they want to pick up, but they are currently holding a baseball bat, so they walk onto their nest so that they can store the bat there.



As seen above the user already has a shield and 2 peas which have been saved in the nest's inventory since before they last logged out. The watermelon they want is also near the nest. They then press the "L" key which transfers the baseball bat from the player's inventory to the nest's inventory, as seen below.



Now that the user's inventory is empty, they press the right arrow twice, then the up arrow once to move onto the tile that the watermelon is on, collecting it and putting it into their inventory, as seen below.

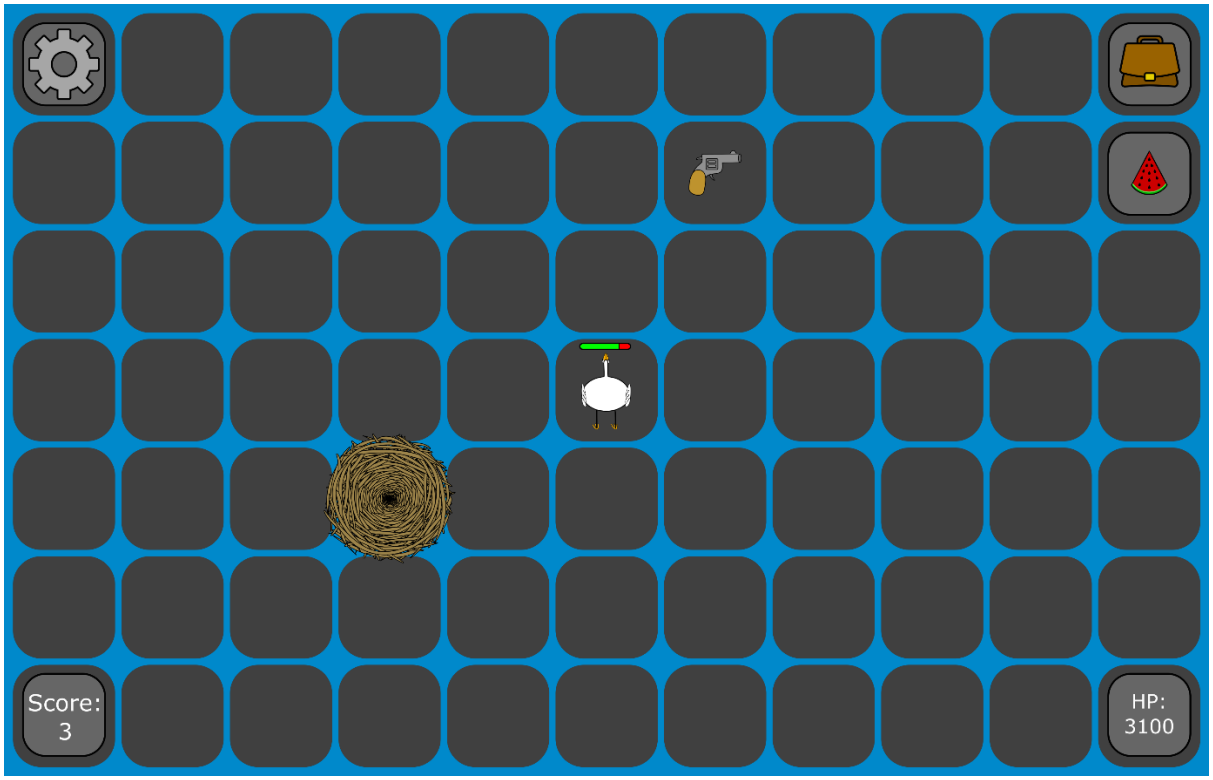


Pressing L moves the held item's ITEM row from the player to the nest (HolderPlayerID = NULL, NestID = <nest id>). Walking onto an item tile does the reverse (TileID = NULL, HolderPlayerID = <player id>) and HeldItemID is updated to match.

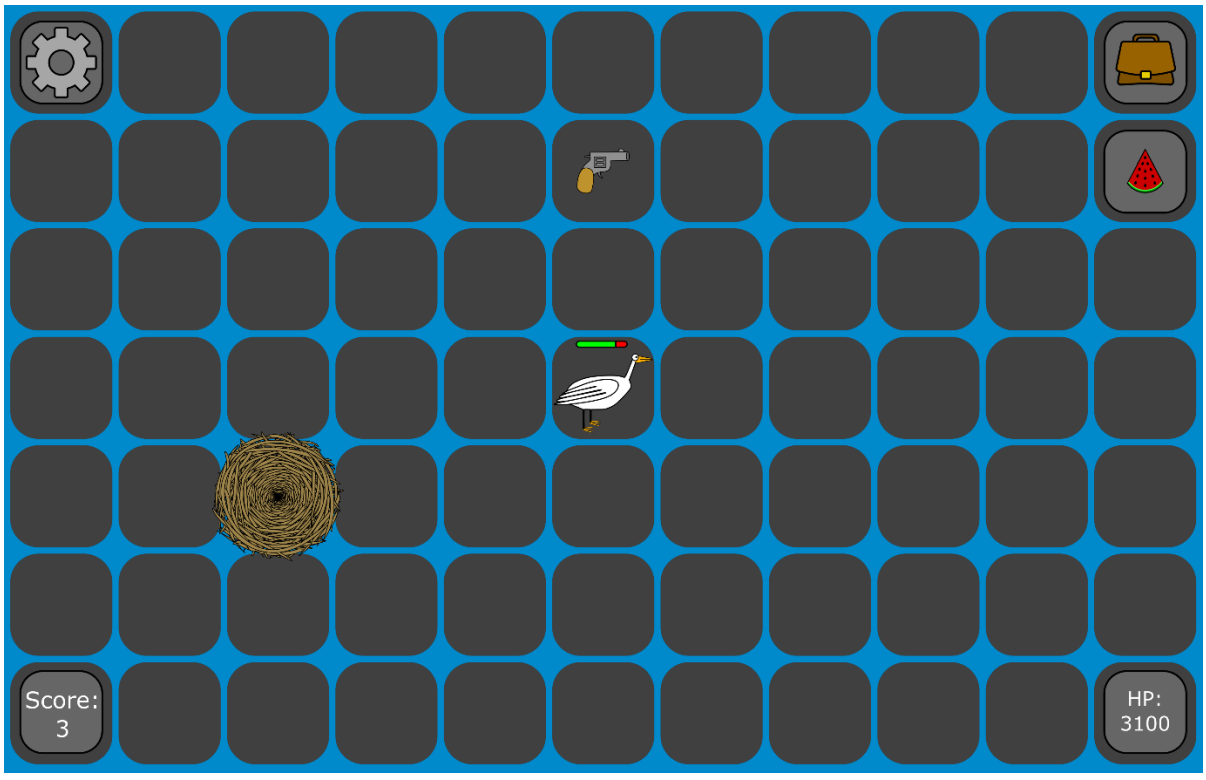
Continue to storyboard 16.

Item spawns, player moves, logs out, logs in, item stays in the same location the whole time, storyboard 16, requirement 10:

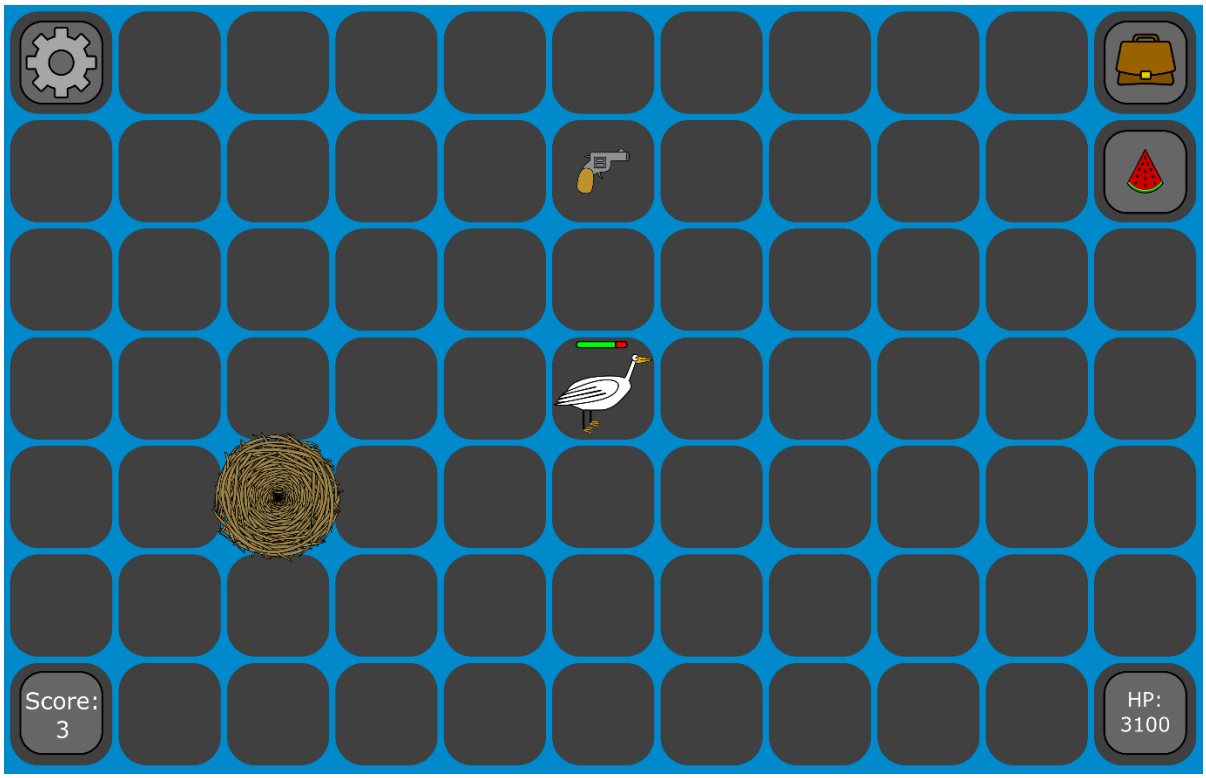
A gun spawns on a tile two tiles above the user and one tile to the right.



The user then clicks the right arrow once, and the gun is now 2 tiles above them, showing that it has stayed in the same location on the map.



The user then clicks the settings icon, bringing them to the settings screen (storyboard 7). They then click the “Return to title screen” button, returning them to the title screen (storyboard 1). At this point there are no players online. They then close the program. They then re-open the program, bringing them to the title screen (storyboard 1). They then type in their username and click the “next” button, bringing them to the login screen (storyboard 2). They enter their password and click the “Login” button, bringing them back to where they were when they logged out, as seen below. The gun is in the same place as when they logged out, showing that the item location persists even upon all online users logging out.



Item position is in ITEM.TileID, which doesn’t depend on any particular player, so it persists even when all PLAYER rows have IsOnline set to 0.

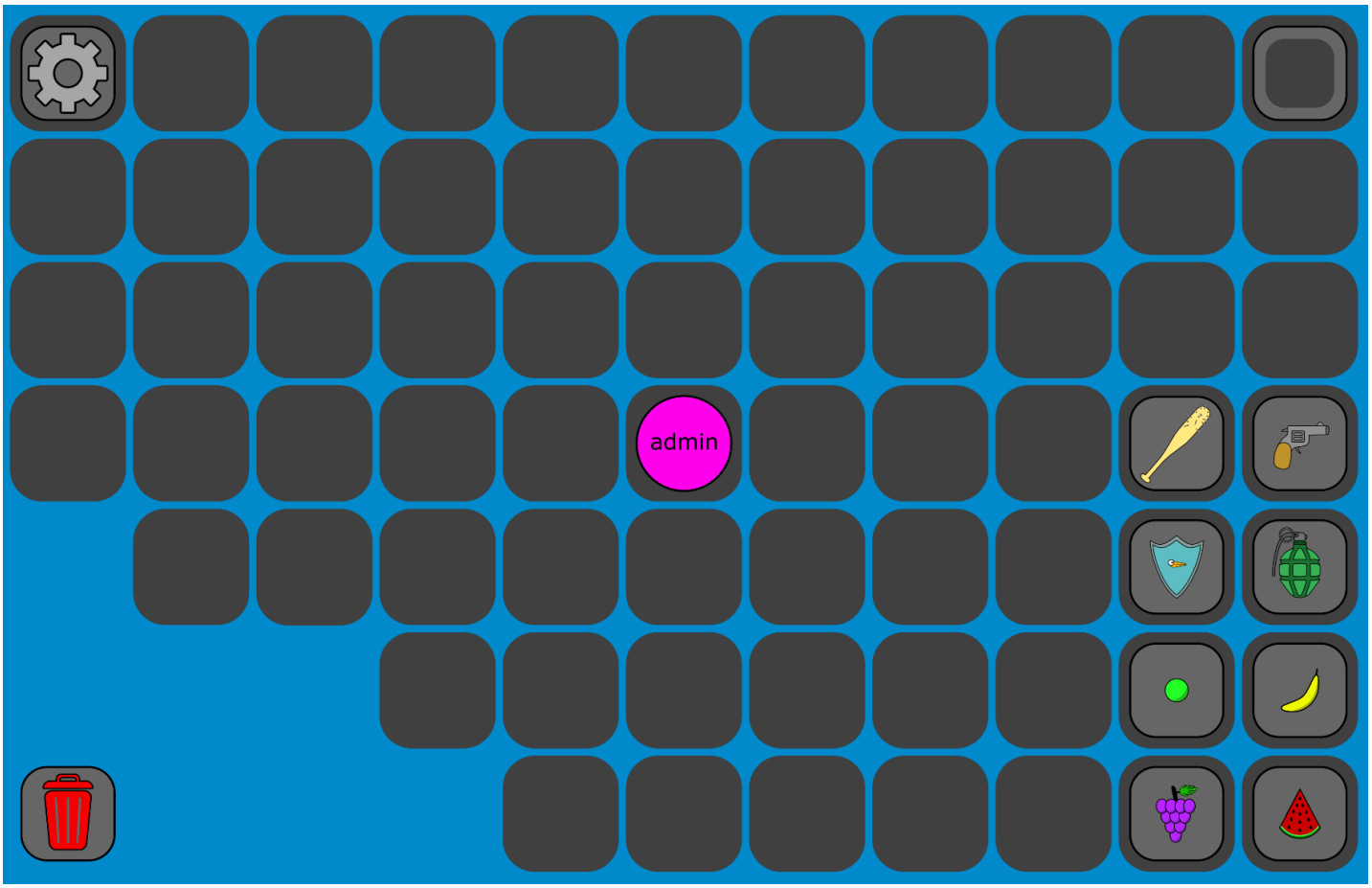
Admin returning to the game, storyboard 17, segway to requirements 4 and 5:

On storyboard 2 the user logs in. If they are an admin they are brought to here. This is the last place that the user (from now on referred to as admin) logged out. If the admin then clicks the settings icon they will be brought to storyboard 8. The admin is able to move with arrow keys or WASD like a normal user, however their sprite will not change. An admin does not have a home nest (if they log in for the first time one will not spawn). If the admin presses the space bar or L or G nothing happens. Instead of the inventory icon in the top right corner there is a tile icon. Instead of a health icon in the bottom right corner there are 8 item/consumable icons (one for each item and consumable). Instead of a score icon on the bottom left there is a rubbish bin icon. If an admin clicks any of these icons, they will be highlighted in green. If the admin clicks the rubbish bin icon, then clicks on a tile with an item or consumable on it, the item or consumable will be deleted. If they instead click on a tile without anything on it, the tile will be deleted.

If the admin clicks the tile icon in the top right corner, they will be brought to storyboard 18.

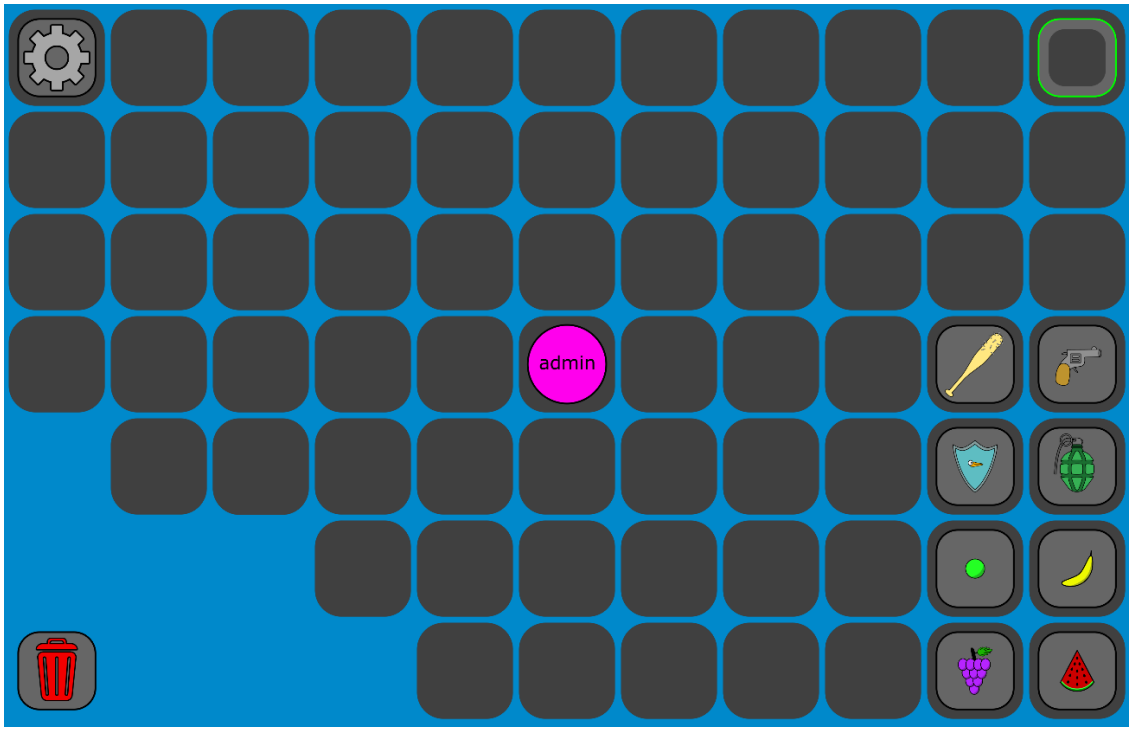
If the admin clicks one of the consumable icons (in this case the banana), they will be brought to storyboard 19.

The admin's PLAYER row has no linked NEST row. The tile/item edit actions write directly to TILE and ITEM.

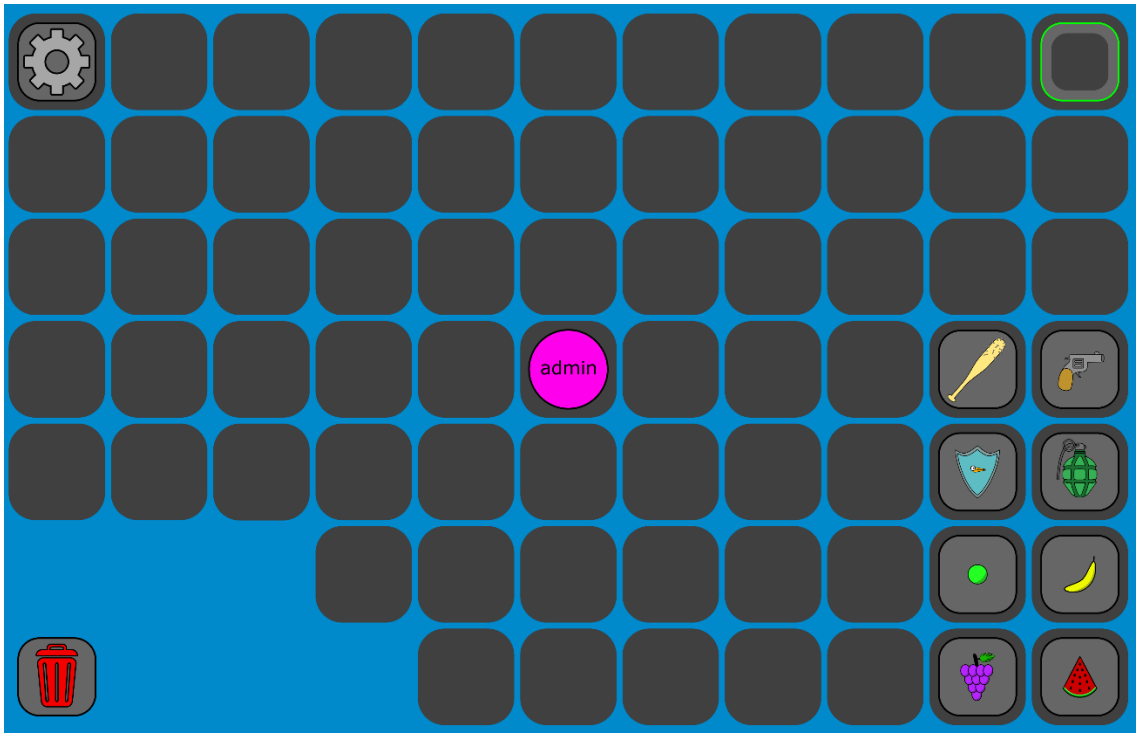


Admin selecting and placing a tile, storyboard 18, requirement 4:

On storyboard 17, the admin clicks the tile icon in the top right corner to select it. The border turns from black to green to indicate that it is selected as seen below.



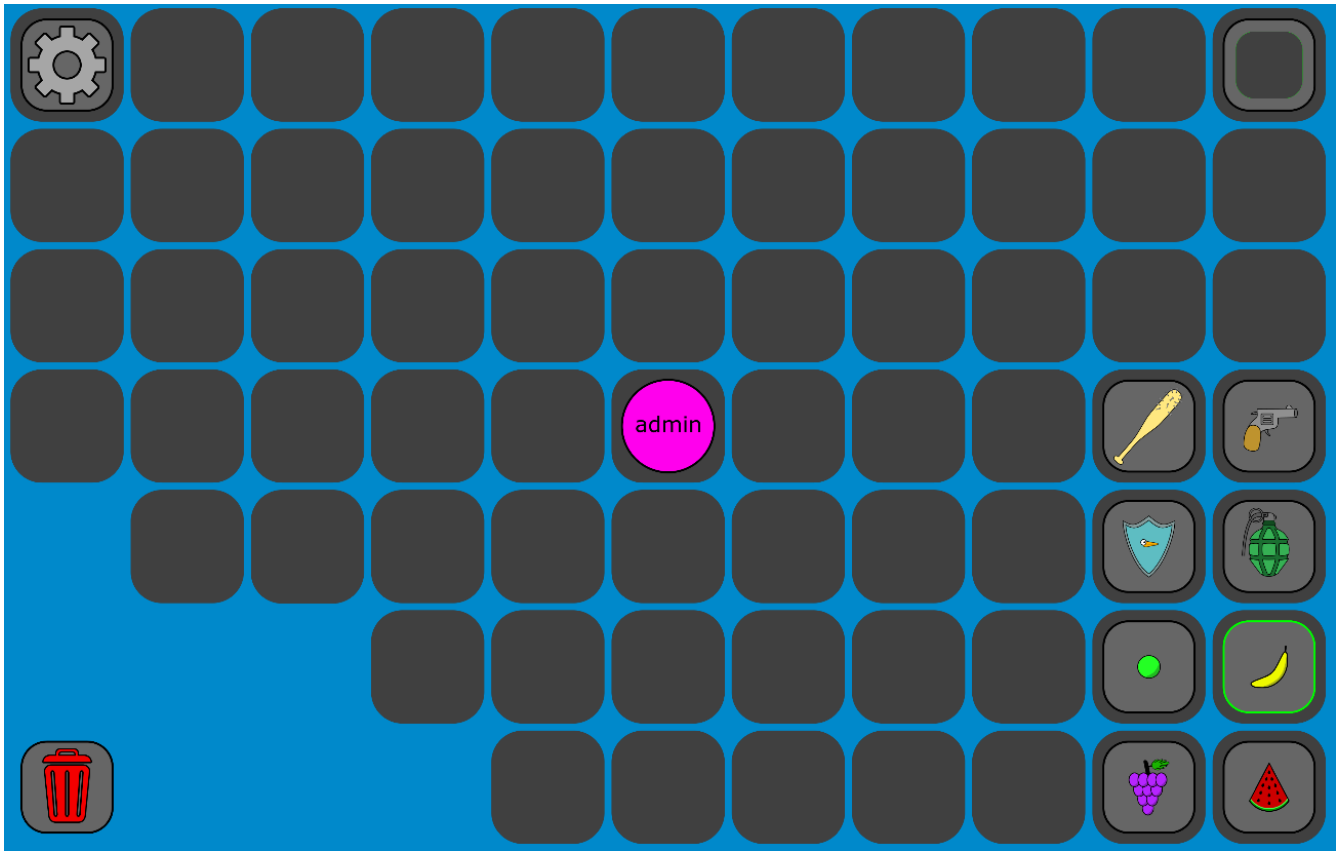
The admin then clicks on the first empty space directly under the settings icon (4 tiles down) to place a tile there. It then appears there for all players and persists even after all players go offline. Tiles can only be placed on empty spaces which are adjacent to another tile (above, below, left or right). The tile placed is shown below.



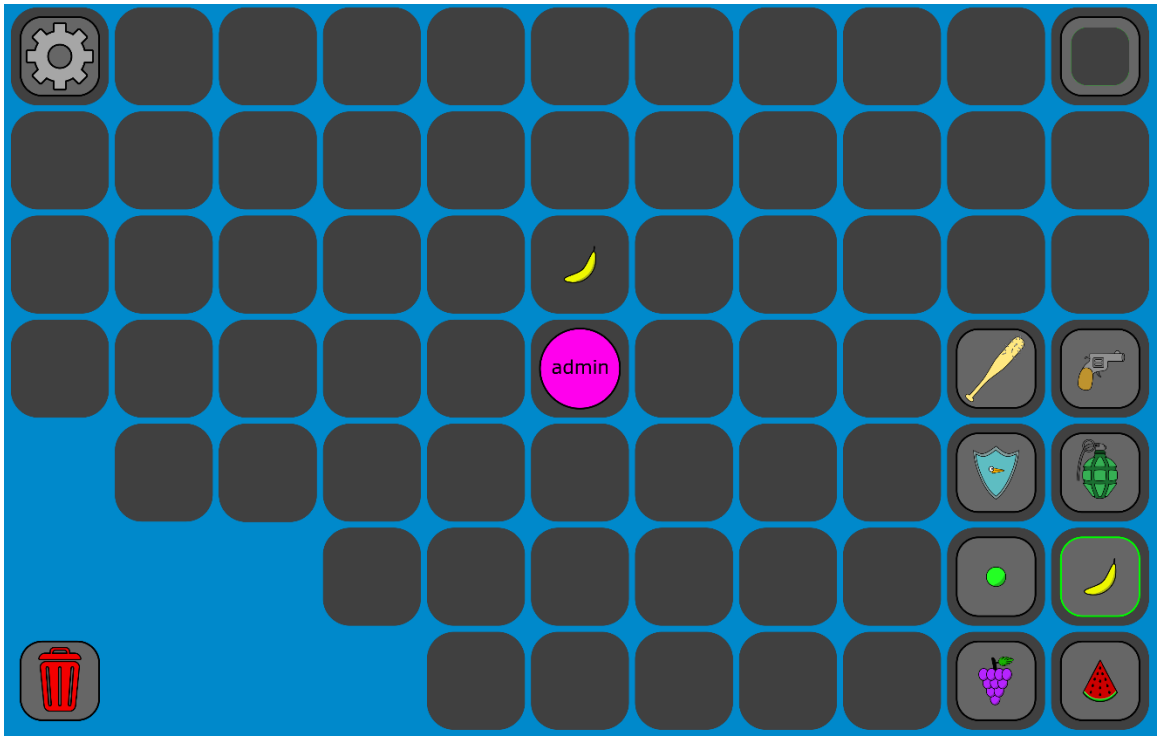
Placing inserts a new TILE row, deleting removes it. Can only do this when no PLAYER, ITEM, or NEST reference the TILE row.

Admin selecting and placing an item on a tile, storyboard 19, requirement 5:

In storyboard 17 the admin selects the banana icon. The border on the icon turns from black to green to indicate that it is selected as seen below.

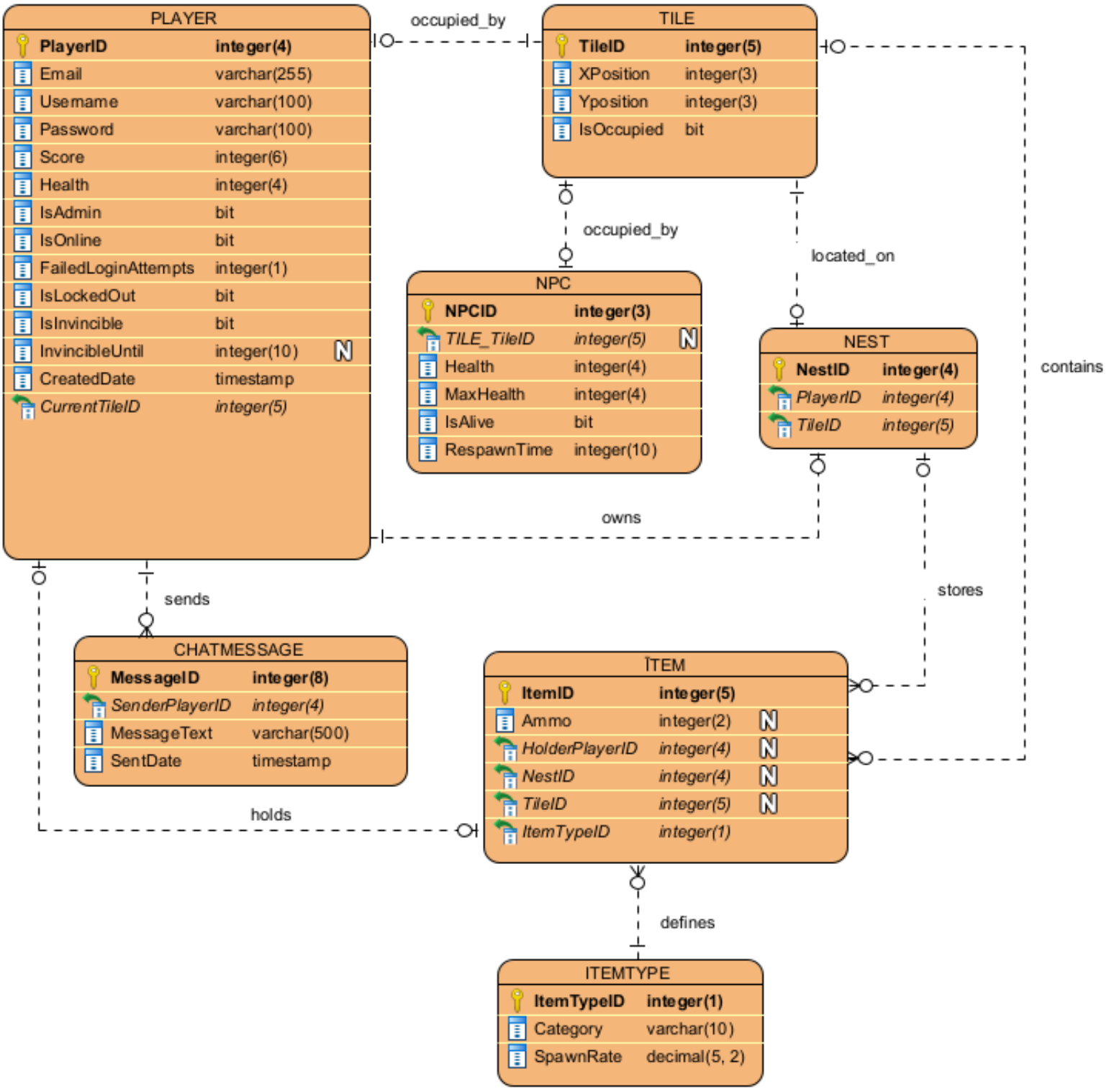


The admin then clicks the empty tile above them (items can only be placed on a tile without an already existing item on it). This places the banana on the tile and it appears for all players. Its location persists even if all players log out. The placed item can be seen below.



Placing inserts a new ITEM row. The rubbish bin icon deletes an ITEM row for a tile that has one.

Task 2, logical entity relationship diagram depicting the planned database:



Task 3, CRUD table as part of analysis for the game system:

Number	Process	Tables involved	Key attributes affected	CRUD operation(s)	Description
1	Player login (including lockout).	PLAYER	Username, Password, FailedLoginAttempts, IsLockedOut, Isoline	Read, Update.	Username/password checked against PLAYER. A failed attempt adds 1 to FailedLoginAttempts. A success resets it to 0. If it gets to 5, IsLockedOut is changed to 1.
2	Player registration.	PLAYER	Email, Username, Password, Score, IsAdmin, IsLockedOut, FailedLoginAttempts, CreatedDate	Create.	A new row is inserted when a player enters an unused username and submits their details, with Score being set to 0 by default.
3	Laying out tiles on the board.	TILE	TileID, XPosition, YPosition	Create, delete.	Admin inserts a new tile next to an existing tile to extend the board, or deletes an unoccupied tile.
4	Placing an item on a tile.	ITEM, ITEMTYPE, TILE	ItemTypeID, TileID	Create, delete.	Either an admin places a new item on an unoccupied tile (adding an ITEM row), or one spawns randomly. The admin bin tool is used to delete an ITEM row from a tile.
5	Player movement.	PLAYER	CurrentTileID	Update	Moving to an adjacent unoccupied tile updates the player's CurrentTileID to the destination tile.
6	Game play scoring.	PLAYER	Score, Health	Update.	Defeating another player increases the attacker's Score by 1. Taking damage reduces the target's health, which then resets upon respawning.
7	Acquiring inventory.	ITEM, NEST	HolderPlayerID, TileID, NestID	Update.	Picking up an item updates its ITEM row so that HolderPlayerID is set and TileID and/or NestID are cleared. Storing it in the nest does the reverse.

8	Item (NPC) movement.	NPC, ITEM	TileID, Health, IsAlive, RespawnTime	Update, Create.	The NPC's TileID is updated each time it moves. On defeat, IsAlive is set to 0 and a new ITEM row is inserted at the tile it died on with a randomly chosen type.
9	Kill running games.	PLAYER	IsOnline	Update	When an admin clicks the "KILL current game" button and confirm, every currently online player's IsOnline flat is set to 0, logging them out and sending them back to the login screen.
10	Add new players (admin).	PLAYER	Email, Username, Password, Score, IsAdmin, CreatedDate	Create.	Same as player registration, but done by an admin from the player adder screen with IsAdmin able to be set.
11	Update data of a player.	PLAYER	Email, Username, Password, Score, IsAdmin, IsLockedOut	Update.	Admin edits an existing player's fields on the player editor screen (with IsAdmin able to be set, and IsLockedOut turned on or off), updating the corresponding PLAYER row.
12	Delete a player.	PLAYER, NEST, ITEM, CHATMESSAGE	Whole PLAYER row, related NEST row, related ITEM rows, SenderPlayerID on CHATMESSAGE.	Delete.	Deleting a player, either by themselves or by an admin, removes their PLAYER row along with their NEST and any ITEM rows they hold or store. Their CHATMESSAGE rows are also removed.

Task 4, DDL SQL script to create tables and relations between tables to support the game:

Note: Added “name” and “description” to ITEMTYPE table.

The DDL SQL script itself is provided as a separate file (it is also in the GitHub repository).

The DDL is implemented as a single T-SQL procedure (usp_CreateGooseFighterDatabase), which builds the entire database from scratch, including all 7 tables with constraints and indexes, and some test data which covers everything in the CRUD table. The script can be run repeatedly without error by checking if each part exists (dropping first if it does) before creating it. It kicks anyone connected to it first before doing this. This is good for repeated testing.

Tables were created in dependency order (because foreign keys cant reference something that doesn’t exist). Every table has a primary key, and foreign keys enforce the relationships from the ERD. UNIQUE constraints enforce other rules (e.g. UQ_Tile_Position stops 2 tiles occupying the same space, UQ_Nest_Player ensures there is only one nest per player, CK_Item_OneLocation makes sure that an item is always on a tile, held by a player or in a nest, etc.).

Only 2 relationships cascade when deleting. Deleting a player removes their nest and chatmessage rows. All other foreign keys use NO ACTION. This is because ITEM is reachable from PLAYER through 2 cascading paths (through HolderPlayerID and through NEST.PlayerID -> NEST -> ITEM.NestID). So there isn’t a single order of deletes. Instead, item deletion is done by the application/procedure logic.

Indexes were made on columns that will be queried often during the gameplay.

5. Create a prototype C# .net application using Visual Studio:

Instructions for how to set up and test the application are in the README in the same folder that this word document is in.